Assignment 2 - Question 2

PCCST303 - Data Structures and Algorithms

Vimal Jyothi Engineering College (Autonomous) | Roll No: 7 | Second Year, Third Semester, CSE A

Problem: Online Bookstore - Binary Search Tree (BST)

An online bookstore stores the ISBN keys **45, 20, 60, 10, 30, 50, 70, 25, 55**. A BST is built by inserting the keys in the given order, traversed in three orders, and searched (keys 25, 55, 90) using BST search and linear search.

1. Input Data

```
ISBN keys (insertion order): 45 20 60 10 30 50 70 25 55
Search keys: 25 55 90
```

2. Source Code (C)

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int key; struct Node *left, *right; } Node;

Node* newNode(int k){ Node* n = malloc(sizeof(Node)); n->key=k; n->left=n->right=NULL; return n; }

Node* insert(Node* r, int k){
    if(!r) return newNode(k);
    if(k < r->key) r->left = insert(r->left, k);
    else r->right = insert(r->right, k);
    return r;
}
void inorder(Node* r){ if(r){ inorder(r->left); printf("%d ", r->key); inorder(r->right);} }
void preorder(Node* r){ if(r){ printf("%d ", r->key); preorder(r->left); preorder(r->right);} }
void postorder(Node* r){ if(r){ postorder(r->left); postorder(r->right); printf("%d ", r->key);} }
int height(Node* r){ if(!r) return -1; int a=height(r->left), b=height(r->right); return (a>b?a:b)+1; }

int bstSearch(Node* r, int k, int* cmp){
    printf("  Path: ");
    while(r){
        (*cmp)++;
        printf("%d ", r->key);
        if(k == r->key){ printf("-> FOUND\n"); return 1; }
        r = (k < r->key) ? r->left : r->right;
    }
    printf("-> NOT FOUND\n");
    return 0;
}
int linearSearch(int a[], int n, int k, int* cmp){
    for(int i=0;i<n;i++){ (*cmp)++; if(a[i]==k) return i; }
    return -1;
}

int main(void){
    int keys[] = {45,20,60,10,30,50,70,25,55};
    int n = sizeof(keys)/sizeof(keys[0]);
    Node* root = NULL;
    for(int i=0;i<n;i++) root = insert(root, keys[i]);

    printf("Inorder   : "); inorder(root);   printf("\n");
    printf("Preorder  : "); preorder(root);  printf("\n");
    printf("Postorder : "); postorder(root); printf("\n");
    printf("Height of BST (edges) = %d\n\n", height(root));

    int targets[] = {25,55,90};
    int totB=0, totL=0;
    printf("%-8s %-14s %-14s\n","Key","BST cmps","Linear cmps");
    for(int i=0;i<3;i++){
        int cb=0, cl=0;
        printf("Searching %d (BST)\n", targets[i]);
        bstSearch(root, targets[i], &cb);
        linearSearch(keys, n, targets[i], &cl);
```

```
        printf("  BST=%d  Linear=%d\n", cb, cl);
        totB+=cb; totL+=cl;
    }
    printf("\nTotal comparisons: BST=%d, Linear=%d\n", totB, totL);
    return 0;
}
```

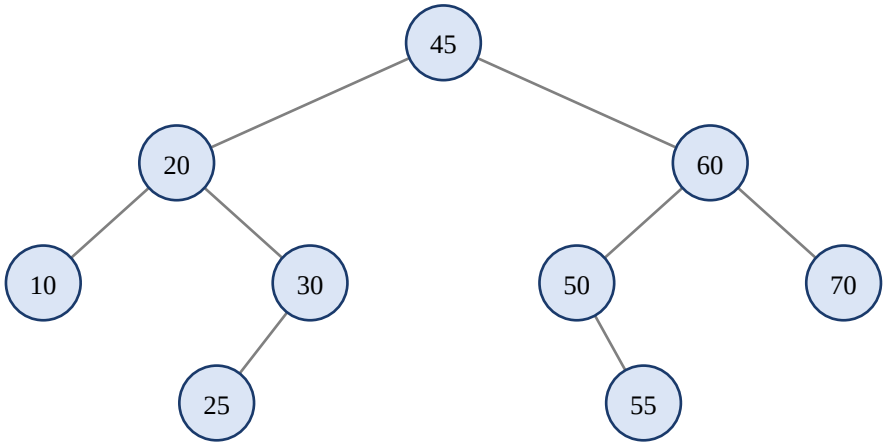
3. Program Output

```
Inorder   : 10 20 25 30 45 50 55 60 70
Preorder  : 45 20 10 30 25 60 50 55 70
Postorder : 10 25 30 20 55 50 70 60 45
Height of BST (edges) = 3

Key      BST cmps       Linear cmps
Searching 25 (BST)
  Path: 45 20 30 25 -> FOUND
  BST=4  Linear=8
Searching 55 (BST)
  Path: 45 60 50 55 -> FOUND
  BST=4  Linear=9
Searching 90 (BST)
  Path: 45 60 70 -> NOT FOUND
  BST=3  Linear=9

Total comparisons: BST=11, Linear=26
```

4. Resulting BST



Height = 3 edges (4 levels), number of nodes n = 9.

5. Traversal Results

Traversal	Result
Inorder	10 20 25 30 45 50 55 60 70 (sorted)
Preorder	45 20 10 30 25 60 50 55 70
Postorder	10 25 30 20 55 50 70 60 45

6. Trace Table - Insertion (key by key)

Step	Key	Path followed from root	Placed as
1	45	(empty tree)	Root
2	20	45 -> left	Left of 45
3	60	45 -> right	Right of 45
4	10	45 -> 20 -> left	Left of 20
5	30	45 -> 20 -> right	Right of 20
6	50	45 -> 60 -> left	Left of 60
7	70	45 -> 60 -> right	Right of 60

Step	Key	Path followed from root	Placed as
8	25	45 -> 20 -> 30 -> left	Left of 30
9	55	45 -> 60 -> 50 -> right	Right of 50

7. Trace Table - Search (comparisons = nodes examined)

Key	BST path	BST cmps	Linear cmps	Result
25	45 -> 20 -> 30 -> 25	4	8 (index 7)	Found
55	45 -> 60 -> 50 -> 55	4	9 (index 8)	Found
90	45 -> 60 -> 70 -> null	3	9	Not found
Total		11	26	

8. Complexity Analysis

	BST Search	Linear Search
Best case	O(1)	O(1)
Average case	O(log n)	O(n)
Worst case	O(n) (skewed tree)	O(n)
Space	O(h) recursive / O(1) iterative	O(1)

A balanced BST with n = 9 has minimum height floor(log2 9) = 3, so this tree is perfectly height-balanced. Insertion costs O(h) per key; building the tree is O(n log n) on average and $O(n^2)$ in the worst case. Traversals are O(n).

9. Comparison Table

Criterion	BST Search	Linear Search
Comparisons for 25	4	8
Comparisons for 55	4	9
Comparisons for 90 (absent)	3	9
Total (3 searches)	11	26
Theoretical bound	O(log n) ~ 3.17 for n=9	O(n) = 9
Extra benefit	Inorder gives sorted keys	None
Extra memory	Two pointers per node	None

10. Analysis of Shape and Height

Each BST search costs at most h + 1 comparisons, so the height of the tree determines search performance. Here the tree is balanced (both subtrees of the root have height 2 from the child level), so every search needs at most 4 comparisons, matching log2(9) = 3.17. Linear search needed 8 or 9 comparisons for the same keys, about 58% more work overall (26 vs 11). If the same ISBNs were inserted in sorted order (10, 20, ..., 70), the tree would degenerate into a chain of height n-1, and BST search would fall to O(n), no better than linear search. This shows that BST efficiency depends on insertion order.

11. Final Conclusion

For the given ISBN dataset, **BST Search is the preferable method**: it needs fewer comparisons (11 vs 26), its observed cost agrees with the theoretical O(log n), and inorder traversal gives the ISBNs in sorted order at no extra cost. For large databases or data that may arrive in sorted order, a self-balancing tree (AVL or Red-Black tree) should be used to guarantee O(log n) search.